

ADR-001 - Component Model and Storage for Combat Vehicles

Field	Value
Purpose & Intent	Decide how combat vehicle customization is represented in code and persisted in the database, balancing flexibility, validation, and balancing velocity.
Incoming	Ch. 1 gameplay flexibility goals, Ch. 2 Rust/PostgreSQL constraints, v1 tech-tree documents in doc/v1.
Outgoing	Ch. 5 building blocks (blueprint and rule engine), Ch. 6 runtime validation/stats pipeline, Ch. 8 data modeling concept.

Status

Accepted (Opt C); amended 2026-07-12 (mount points and assembly slots)

Date

2026-06-27 (original); 2026-07-12 (mount-point amendment)

Context

The current schema stores fixed combat stats in `basic_tank`, which is suitable for one vehicle archetype but does not scale to composable builds (`chassis -> mount -> turret -> weapon -> utility`) and compatibility rules defined in v1 documentation.

We need:

- Data-driven expansion of components without code rewrites for every new part.
- Dedicated properties for specific component kinds (for example turret rotation speed).
- Fast and explainable validation for requirements, incompatibilities, and balancing rules.

Decision

Use a hybrid component model (Opt C):

- Generic component envelope for shared fields and rule graph behavior.
- Typed component payload per component kind for dedicated properties.
- Data-driven compatibility and dependency rules (tags + explicit rules), not inheritance trees.

For storage, use a primarily normalized schema with optional JSONB extension fields for low-frequency experimental attributes.

Options Considered

Option A - Deep class hierarchy with subclass-specific properties

Description:

- Represent components mainly through class inheritance (for example `Component -> Mount -> Turret`).

Pros:

- Strong compile-time typing in code.
- Simple when component set is small and stable.

Cons:

- Rigid and expensive to evolve for content-heavy systems.

- Encourages logic spread across subclasses instead of centralized rule evaluation.
- Harder to let designers/balance work mostly through data.

Decision:

- Rejected.

Option B - Fully generic JSON document for all components

Description:

- Keep all component fields in JSONB with minimal relational structure.

Pros:

- Very flexible and quick for experimentation.
- Few migrations required for new fields.

Cons:

- Weak constraints and referential integrity.
- Harder query performance and indexing for gameplay-critical paths.
- Increased risk of runtime validation bugs.

Decision:

- Rejected as primary model.

Option C - Normalized core + typed details + optional JSONB extensions

Description:

- Shared fields in relational tables.
- Kind-specific detail tables for dedicated properties.
- Optional `extra_json` for experimental attributes.

Pros:

- Strong integrity for ids, slots, tags, compatibility, and progression queries.
- Clear separation between shared and dedicated fields.
- Extensible without large inheritance refactors.

Cons:

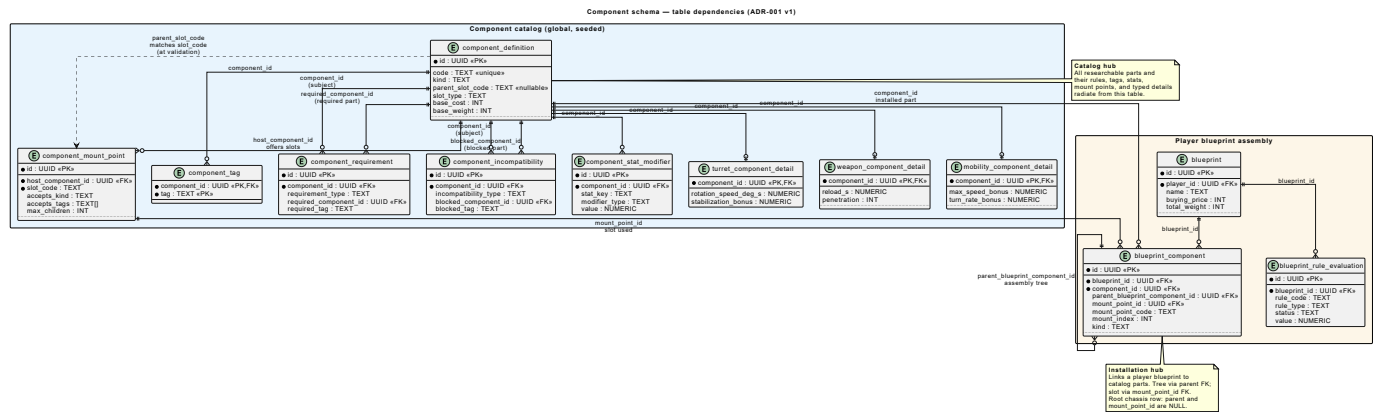
- More tables and migrations than pure JSON.
- Slightly higher upfront modeling effort.

Decision:

- Accepted.

Recommended Schema (v1)

Schema dependency diagram



Component Catalog (researchable parts)

- **component_definition**
 - **id** UUID PK
 - **code** TEXT UNIQUE
 - **name** TEXT
 - **kind** TEXT (chassis, mount, turret, weapon, mobility, utility, armor, power)
 - **parent_slot_code** TEXT NULL — slot on the parent host this component expects (for example `primary_turret`, `main_weapon`); NULL for root chassis
 - **slot_type** TEXT — role this component plays when installed (synonym for **kind** in early v1; may diverge later)
 - **base_cost** INT
 - **base_weight** INT
 - **power_draw** INT
 - **power_supply** INT
 - **extra_json** JSONB NULL
- **component_mount_point**
 - **id** UUID PK
 - **host_component_id** UUID FK -> **component_definition.id** — catalog row that **offers** the slot (for example Tank chassis)
 - **slot_code** TEXT — stable slot identifier within the host (for example `primary_turret`, `main_weapon`)
 - **accepts_kind** TEXT — component **kind** allowed in this slot (for example `turret`, `weapon`)
 - **accepts_tags** TEXT[] NULL — optional tag filter (for example `{turret-capable}`)
 - **max_children** INT NOT NULL DEFAULT 1 — how many components may occupy this slot on one blueprint
 - **required** BOOLEAN NOT NULL DEFAULT false — whether the slot must be filled for a complete build
 - **UNIQUE** (host_component_id, slot_code)
- **component_tag**
 - **component_id** UUID FK -> **component_definition.id**
 - **tag** TEXT
 - PK (component_id, tag)
- **component_requirement**
 - **id** UUID PK
 - **component_id** UUID FK
 - **requirement_type** TEXT (component, tag)
 - **required_component_id** UUID NULL FK
 - **required_tag** TEXT NULL
- **component_incompatibility**

- `id` UUID PK
- `component_id` UUID FK
- `incompatibility_type` TEXT (component, tag)
- `blocked_component_id` UUID NULL FK
- `blocked_tag` TEXT NULL
- `component_stat_modifier`
 - `id` UUID PK
 - `component_id` UUID FK
 - `stat_key` TEXT
 - `modifier_type` TEXT (flat, percent)
 - `value` NUMERIC

Typed Detail Tables (dedicated properties)

- `turret_component_detail`
 - `component_id` UUID PK FK -> `component_definition.id`
 - `rotation_speed_deg_s` NUMERIC
 - `stabilization_bonus` NUMERIC
- `weapon_component_detail`
 - `component_id` UUID PK FK
 - `reload_s` NUMERIC
 - `penetration` INT
 - `alpha_damage` INT
 - `dispersion` NUMERIC
- `mobility_component_detail`
 - `component_id` UUID PK FK
 - `max_speed_bonus` NUMERIC
 - `turn_rate_bonus` NUMERIC
 - `terrain_penalty_factor` NUMERIC

Player Blueprint Assembly

- `blueprint`
 - keep existing id/name/player link
 - add computed summary fields as needed (`buying_price`, `total_weight`, `power_balance`)
- `blueprint_component`
 - `id` UUID PK
 - `blueprint_id` UUID FK -> `blueprint.id`
 - `component_id` UUID FK -> `component_definition.id`
 - `parent_blueprint_component_id` UUID NULL FK -> `blueprint_component.id` — installed parent in the assembly tree
 - `mount_point_id` UUID NULL FK -> `component_mount_point.id` — catalog slot occupied on the parent (NULL for root chassis)
 - `mount_point_code` TEXT NULL — optional denormalized snapshot of `component_mount_point.slot_code` at install time
 - `mount_index` INT NULL — disambiguates multiple children in the same slot when `max_children > 1`
 - denormalized snapshot fields as today (`kind`, `game_image_url`, `menu_image_url`)

Rule Evaluation (optional explicit rule ids)

- `blueprint_rule_evaluation`
 - `id` UUID PK
 - `blueprint_id` UUID FK
 - `rule_code` TEXT
 - `rule_type` TEXT (`hard`, `soft`, `negative`)
 - `status` TEXT (`pass`, `fail`, `penalty`, `bonus`)
 - `value` NUMERIC NULL

Schema References (mount-point graph)

`component_mount_point` is catalog data only — it is **not** player-owned. References:

From	To	Link type	Purpose
<code>component_mount_point.host_component_id</code>	<code>component_definition.id</code>	FK	Host that offers the slot (for example Tank chassis)
<code>component_definition.parent_slot_code</code>	<code>component_mount_point.slot_code</code>	logical match	Child declares which slot code it expects on its parent
<code>blueprint_component.mount_point_id</code>	<code>component_mount_point.id</code>	FK	Which catalog slot this installation occupies on the parent
<code>blueprint_component.parent_blueprint_component_id</code>	<code>blueprint_component.id</code>	FK	Installed parent row in the blueprint tree
<code>blueprint_component.component_id</code>	<code>component_definition.id</code>	FK	Installed part from the catalog

Validator lookup at install time (no extra FK required):

```
child.parent_slot_code
→ find component_mount_point on parent.component_definition
  where slot_code = parent_slot_code
```

- verify accepts_kind / accepts_tags / max_children
- insert blueprint_component.mount_point_id

Empty-slot UI lookup:

```
blueprint chassis row → component_definition
  → component_mount_point rows for host_component_id
  → subtract slots already used by child blueprint_component rows (via mount_point_id)
```

mount_point_code on blueprint_component is an optional denormalized snapshot of component_mount_point.slot_code for query convenience; mount_point_id is the canonical reference.

```
component_definition (Tank chassis)
├─ component_mount_point (primary_turret)
│   └─ referenced at install by blueprint_component.mount_point_id
│       └─ child blueprint_component (Scout turret)
│           parent_blueprint_component_id → chassis row
│           component_id → Scout turret definition
│           parent_slot_code on child definition matched slot_code at validation
```

Amendment: Mount Points and Assembly Slots (2026-07-12)

Context

Composable builds require hosts to **offer** attachment slots and child components to declare **where** they attach. The v1 roadmap story (*buy turret and light machine gun, mount on Tank/Truck blueprint*) needs this before compatibility tags and stat rules can be evaluated meaningfully.

Two related concepts:

Concept	Stored on	Question answered
Mount point	Host component_definition via component_mount_point	"What sockets does this part offer?"
Parent slot expectation	Child component_definition.parent_slot_code	"Which socket does this part expect?"

Installed structure is recorded on blueprint_component through parent_blueprint_component_id and mount_point_id (FK to component_mount_point). Rule **evaluation timing and outcomes** remain in ADR-002; this amendment defines only the persistent model.

Options Considered

Mount Opt A — Implicit slots via kind + parent_slot_code only

- No component_mount_point table; validity inferred from parent/child kind and tags.
- **Rejected as sole model** — insufficient for slot counts, Blueprint Workshop empty-slot UI, and explainable "slot full" errors.

Mount Opt B — Explicit mount points on host definitions

- component_mount_point rows on catalog hosts; children declare parent_slot_code.
- **Accepted** — data-driven slot layout, aligns with normalized Opt C, supports UI and validator.

Mount Opt C — Global slot taxonomy

- Shared slot-type registry; components link via `provides_slot` / `requires_slot`.
- **Rejected for v1** — extra indirection without enough chassis variety yet.

Mount Opt D — JSONB mount config on `component_definition`

- `extra_json.mountPoints` on hosts only.
- **Rejected as primary model** — acceptable for prototyping, but weaker integrity; may be used temporarily during migration only.

Decision

Adopt **Mount Opt B** with a phased rollout:

1. **v1:** define `component_mount_point` on **chassis** hosts only (Tank, Truck). Turret and weapon are children located by `parent_slot_code`.
2. **Later:** allow any installed host (for example turret, mount ring) to offer child slots via additional `component_mount_point` rows.

Mount compatibility is a **hard rule** checked at assembly time (see ADR-002). Validator inputs: blueprint assembly tree + `component_mount_point` catalog + child `parent_slot_code` + tags/requirements.

v1 Slot Layout (seed target)

Host	slot_code	accepts_kind	max_children	Notes
Tank chassis	<code>primary_turret</code>	<code>turret</code>	1	Requires chassis tag <code>turret-capable</code>
Truck chassis	<code>open_mount</code>	<code>turret</code>	1	Open-top; may use tag <code>open-top</code>
Scout turret (child)	—	—	—	<code>parent_slot_code</code> = <code>primary_turret</code> or <code>open_mount</code>
Scout turret (host, phase 2)	<code>main_weapon</code>	<code>weapon</code>	1	Offers weapon socket on installed turret
Light MG	—	—	—	<code>parent_slot_code</code> = <code>main_weapon</code>

For v1, the weapon may attach to a turret row even before `main_weapon` mount points are seeded on turret definitions, as long as the validator enforces `parent_slot_code` against the parent's offered slots (add turret `main_weapon` mount point in the same migration as the turret ticket).

Assembly Example

```
blueprint: "Tank #1"
├─ blueprint_component (chassis: Tank)      parent: NULL, mount_point_id: NULL
│   └─ blueprint_component (turret: Scout)   parent: chassis row, mount_point_id: →
│       primary_turret
│           └─ blueprint_component (weapon: Light MG) parent: turret row, mount_point_id: →
│               main_weapon
```

Install-Time Rules (structural, hard)

When mounting a component onto a blueprint:

1. Resolve parent `blueprint_component` (chassis for turret; turret for weapon in v1).
2. Find `component_mount_point` on the parent's catalog definition matching child's `parent_slot_code`.

3. Verify `accepts_kind` (and optional `accepts_tags`) against the child definition.
4. Count existing children in that slot; fail if `>= max_children`.
5. Insert `blueprint_component` with `parent_blueprint_component_id`, `mount_point_id`, optional `mount_point_code` snapshot, and denormalized snapshot fields.

Consequences of Amendment

Positive:

- Blueprint Workshop can list empty slots from data instead of hard-coded chassis logic.
- Slot-full and wrong-parent failures are explainable and testable.
- Same model scales to mount rings, utility rails, and multi-weapon layouts.

Negative:

- Additional seeding per host component.
- Slightly more complex install API than flat `blueprint_component` rows.

Why This Fits Tank Wars

- Matches v1 graph-based assembly and tag-based compatibility.
- Supports domain-specific stats like turret rotation speed without class explosion.
- Keeps balancing mostly in data while preserving SQL constraints and queryability.

Consequences

Positive:

- Faster content expansion and balancing iterations.
- Better testability: validator and stat aggregation can be deterministic over table data.
- Easier UI explainability for invalid builds and trade-offs.

Negative:

- More migration and data-seeding work in early development.
- Requires disciplined rule validation service boundaries.

Migration Notes from Current Schema

Current state contains fixed columns in `basic_tank` for speed, turret speed, and machine-gun fields.

Incremental migration path:

1. Introduce `component_definition` and `blueprint_component` tables.
2. Create seed components equivalent to existing `basic_tank` capabilities.
3. **Add `component_mount_point`, `parent_slot_code`, and assembly fields on `blueprint_component` (2026-07-12 amendment).**
4. Seed v1 chassis slots and turret/weapon definitions for the mount-component roadmap story.
5. Build a validator + stat aggregator service in backend.
6. Backfill existing blueprint data into component assemblies (chassis rows get `parent_blueprint_component_id = NULL`).
7. Deprecate and remove `basic_tank` after parity and verification.

Validation and Testing Impact

- Add unit tests for requirement/incompatibility evaluation.
- Add unit tests for mount-point validation (`slot_available`, `accepts_kind`, `accepts_tags`, `max_children`).
- Add integration tests for blueprint assembly persistence and stat aggregation.

- Add fixture-based balance tests for representative archetypes (raider, brawler, sniper).

Open Questions

- Which stats should remain persisted as denormalized cache versus computed at runtime?
- Do we need versioned component definitions for live-balance patches?
- Should soft and negative rules always be precomputed or partially runtime-evaluated?
- Should Truck `open_mount` and Tank `primary_turret` share one `parent_slot_code` on turret definitions, or use separate codes with a mapping table?
- When should mount points on non-chassis hosts (turret → weapon) be introduced relative to the v1 turret ticket?